CI/CD Pipeline Design Exercise

CarbonSense

AI-Based Personal Carbon Footprint Tracking and Sustainability Advisor

Submitted in partial fulfilment for the course of
CSA1012 — Software Engineering

towards the award of the degree of
BACHELOR OF TECHNOLOGY

in Artificial Intelligence and Data Science

Submitted by
Deepak Kumar A
192524422

Under the Supervision of
Dr. K. Anitadavamani

1. Introduction and CI/CD Requirements Analysis

1.1 Project Overview

CarbonSense is a web-based application that allows individuals to log daily activities across travel, electricity, food, and waste categories, automatically calculates the resulting carbon dioxide equivalent (CO2e) footprint, and returns rule-based sustainability recommendations. The system is composed of a static HTML/CSS/JavaScript frontend and a Python Flask REST API backend backed by SQLite, with both components containerized using Docker and orchestrated through Docker Compose behind an nginx reverse proxy.

1.2 Technology Stack

Layer	Technology
Frontend	HTML, CSS, vanilla JavaScript
Backend	Python, Flask, gunicorn
Database	SQLite
Containerization	Docker, Docker Compose, nginx
Source Control	Git / GitHub
Existing Tests	pytest (test_carbonsense.py)

1.3 Why CI/CD is Needed for This Project

Analysis of the project structure and problem statement surfaces the following concrete needs for continuous integration and continuous delivery:

- Two independently versioned components (frontend and backend) must be built, tested, and deployed together without manual coordination errors.
- A pytest test suite already exists (test_carbonsense.py) but is only useful if it is run automatically on every change, not manually before submission.
- The application is fully containerized (Dockerfile per service, docker-compose.yml) — this maps naturally onto an automated build-and-package pipeline stage.

- Manual deployment is error-prone and not repeatable; an automated, environment-aware deployment process removes this risk and demonstrates industry-standard practice.
- Carbon calculation logic (carbon_calculator.py) and recommendation logic (recommendation.py) are business-critical — regressions here silently produce incorrect footprints or advice, making automated regression testing essential.
- As an academic deliverable, the pipeline must also be observable and explainable — every stage needs clear tooling justification, quality gates, and rollback safety.

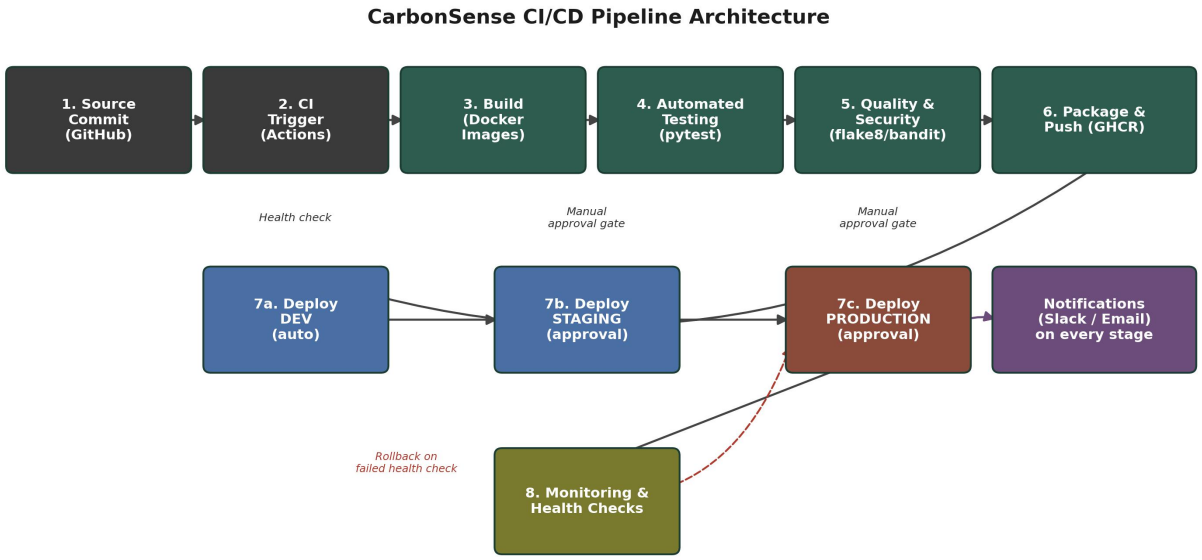
1.4 Derived CI/CD Requirements

Requirement	Description
Automated build	Every push must build Docker images for frontend and backend without manual steps.
Automated testing	Unit and integration tests must run on every commit and block merge on failure.
Code quality & security gate	Static analysis (style + security) must run before code is packaged.
Versioned artifact packaging	Docker images must be tagged and pushed to a registry for traceable deployment.
Multi-environment deployment	Separate, isolated Development, Staging, and Production environments.
Safety mechanisms	Health checks, manual approval gates, rollback, and notifications.

2. CI/CD Pipeline Workflow Design

2.1 Pipeline Architecture Diagram

The diagram below shows the complete CarbonSense CI/CD pipeline from source commit through to production deployment, including monitoring and rollback.



2.2 Branching Strategy

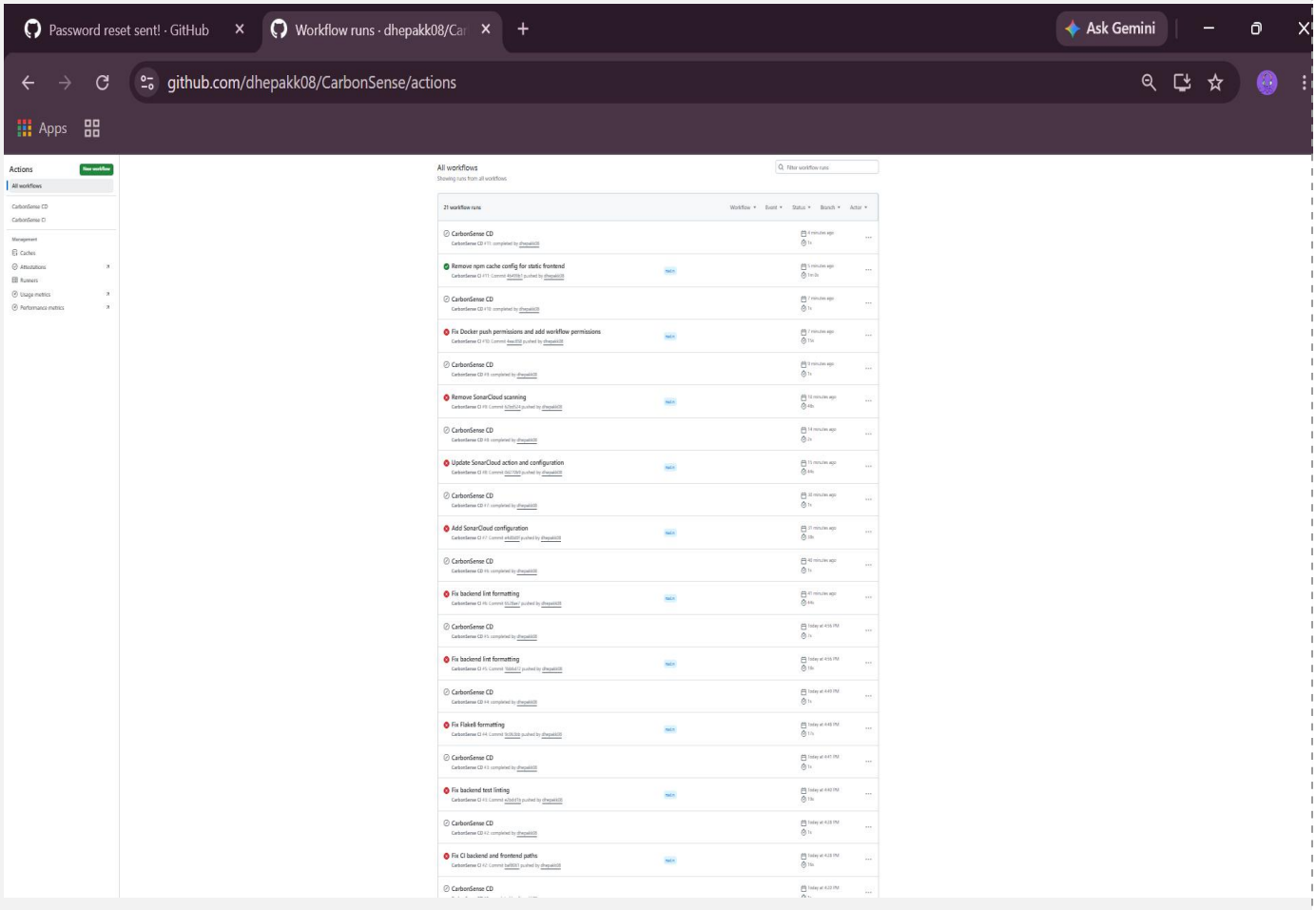
A lightweight two-branch strategy is used, suited to the scale of an academic project while still demonstrating good practice:

- feature/* branches — individual features or fixes, merged into dev via pull request.
- dev branch — integration branch; every push triggers the CI workflow and auto-deploys to the Development environment.
- main branch — production-ready branch; merges here (via reviewed PR) trigger CI, then the CD workflow through Staging and Production with manual approval gates.

2.3 Pipeline Stages Overview

#	Stage	Trigger	Purpose
1	Source Code Management	git push / pull request	Version control, branch protection, code review.
2	CI Trigger	Push or PR to dev/main	GitHub Actions workflow starts automatically.
3	Build	After trigger	Install dependencies, build Docker images for both services.
4	Automated Testing	After build	Run pytest unit and integration tests with coverage.

#	Stage	Trigger	Purpose
5	Code Quality & Security	After tests pass	flake8 style checks and bandit security scan.
6	Packaging	After quality gate passes	Tag and push Docker images to GitHub Container Registry.
7	Deployment	After successful CI run	Deploy to Development, then Staging, then Production.
8	Monitoring & Rollback	Post-deployment	Health checks; automatic rollback on failure.

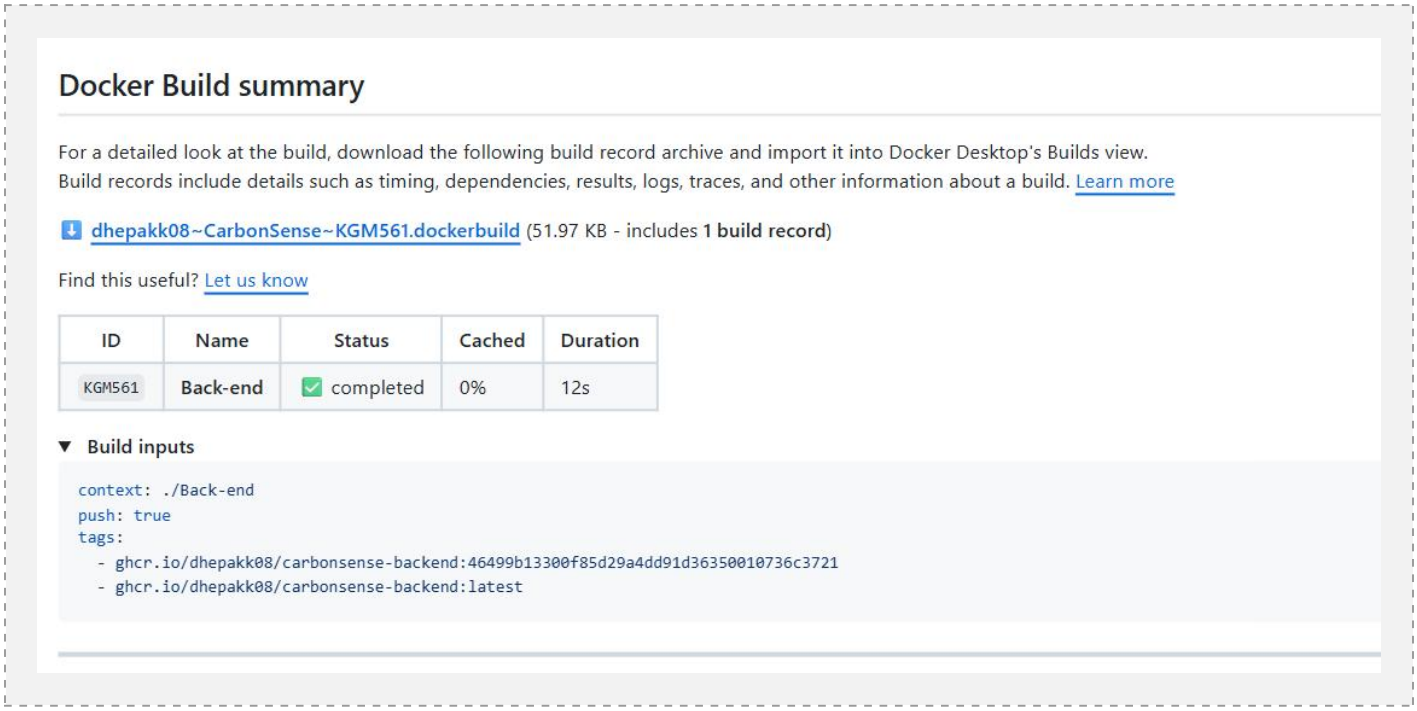


3. Tools and Technologies — Selection and Justification

Tool choices prioritise native integration with the existing GitHub-hosted, Docker-based CarbonSense stack, low setup overhead appropriate for an academic project, and zero additional cost.

Pipeline Stage	Tool	Justification
Source Control & CI Orchestration	GitHub Actions	Native to the existing GitHub repository; no separate CI account or billing needed; pipeline is defined as version-controlled YAML alongside the code.
Backend Testing	pytest, pytest-cov	The project already ships test_carbonsense.py written for pytest; reusing it avoids introducing a second test framework.
Backend Linting	flake8	Lightweight, fast PEP 8 checker with negligible pipeline overhead — fails fast on style issues before deeper stages run.
Backend Security Scanning	bandit	Purpose-built static analyzer for Python that flags real risks (hardcoded secrets, unsafe subprocess/eval usage) without requiring an external SaaS account.
Frontend Linting	ESLint	Industry-standard linter for the vanilla JavaScript in frontend/script.js; catches errors before they reach containers.
Containerization	Docker, Docker Buildx	Matches the project's existing Dockerfiles and docker-compose.yml — the pipeline builds the same images developers already use locally.
Artifact Registry	GitHub Container Registry (GHCR)	Authenticates automatically with the built-in GITHUB_TOKEN — no extra registry account, credentials, or secrets to manage.
Deployment Orchestration	Docker Compose	Reused directly from the project's own docker-compose.yml across Dev, Staging, and Production, keeping environments consistent.

Pipeline Stage	Tool	Justification
Reverse Proxy	nginx	Already used in the project for routing; retained in Production to serve the frontend and proxy API requests.
Environment Approval Gates	GitHub Environments	Built-in required-reviewer protection rules provide manual approval for Staging/Production without a third-party approval tool.
Notifications	Slack / Email Webhooks	Simple webhook step posts pipeline status without needing a dedicated monitoring platform.



4. Automated Testing Strategy

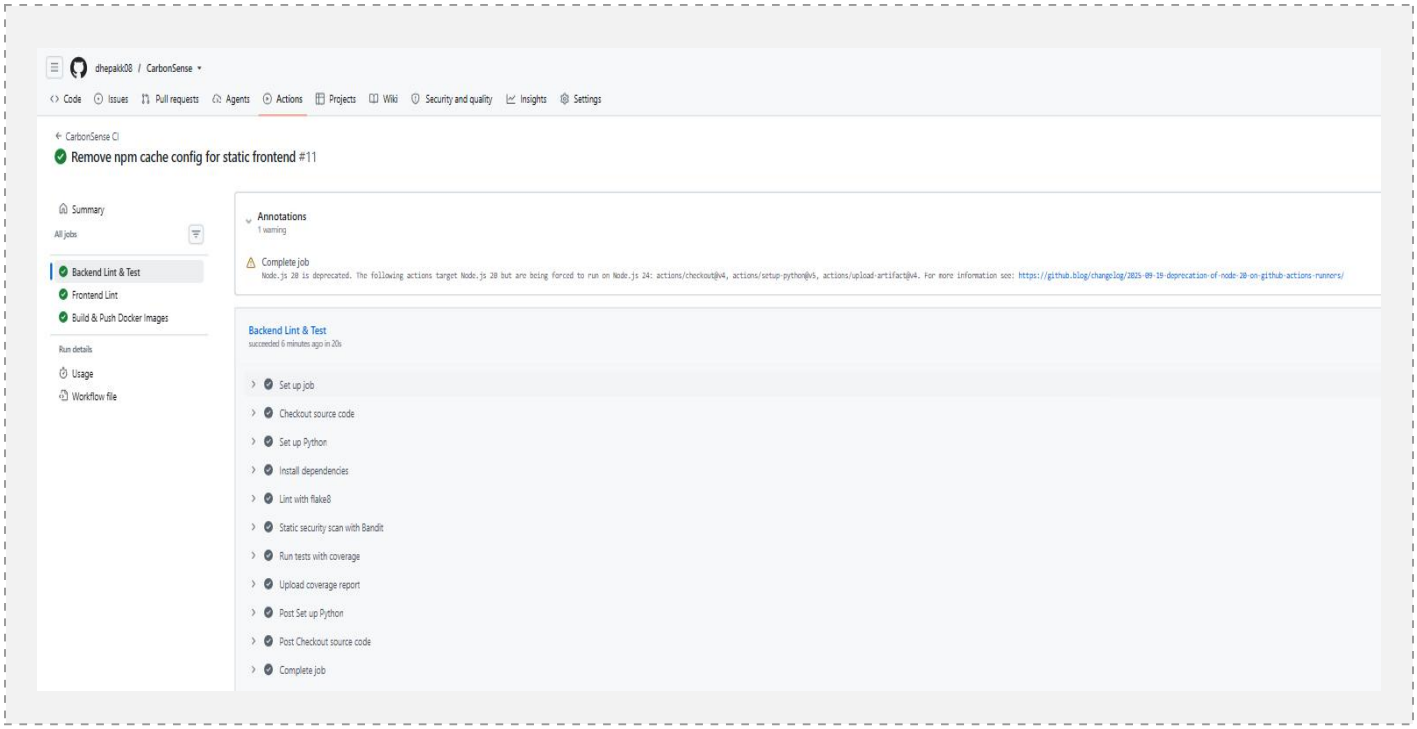
4.1 Test Levels

Test Type	Scope	Tooling
Unit Tests	carbon_calculator.py and recommendation.py pure logic — CO2e formulas per activity	pytest

Test Type	Scope	Tooling
	category, edge cases (zero activity, extreme values, unknown category).	
Integration / API Tests	Flask endpoints exercised end-to-end against a temporary SQLite database: /api/login, /api/activity, /api/activities/<id>, /api/dashboard/<id>, /api/goal, /api/recommendations.	pytest + Flask test client
Static Analysis (style)	Backend Python code conventions and simple correctness issues (unused imports, undefined names).	flake8
Static Analysis (security)	Hardcoded secrets, unsafe function usage, insecure defaults.	bandit
Frontend Linting	Basic syntax and code-quality checks on script.js.	ESLint
Smoke Test (post-deploy)	GET /api/health returns 200 in each environment after deployment.	curl in CD workflow

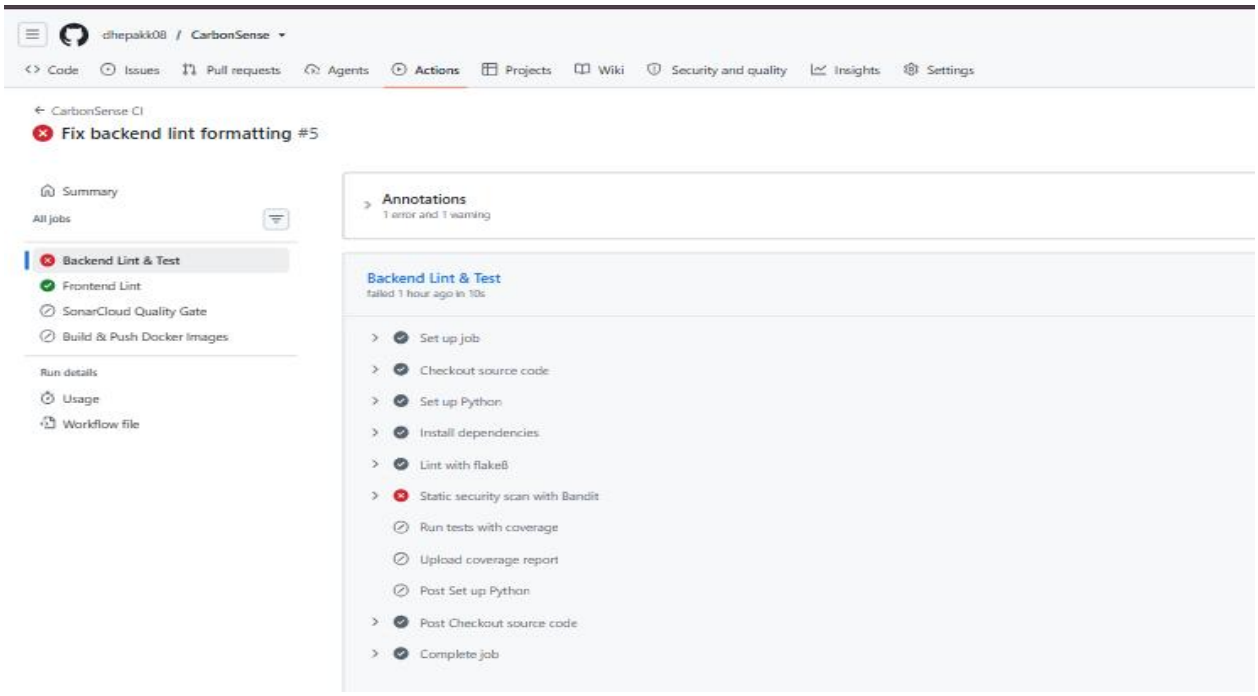
4.2 Coverage and Quality Gate

pytest-cov generates a coverage report on every run, uploaded as a build artifact for traceability. The CI workflow is configured to fail the pipeline if any test fails, or if flake8/bandit report blocking issues — meaning untested or non-compliant code cannot progress to the packaging stage. A practical coverage target of 70% is recommended for this project's size.



4.3 Frontend Testing — Current Scope and Future Work

Given the project's academic scope, frontend verification is currently limited to ESLint static checks. A natural extension would be Selenium or Playwright browser-based smoke tests covering the activity-logging form and dashboard rendering; this is noted as a future improvement rather than implemented in the current pipeline.



5. Deployment Strategy

5.1 Environment Overview

Environment	Trigger	Approval	Purpose
Development	Auto, on every push to dev	None	Fast feedback for active development; latest code always running.
Staging	Auto, after CI passes on main	One required reviewer (GitHub Environment protection rule)	Pre-production validation and demo/QA sign-off before release.
Production	Manual trigger after staging succeeds	Separate required reviewer/approver	Stable, tagged release serving real users; protected by health checks.

5.2 Deployment Mechanics

Each environment runs the same docker-compose.yml used locally, with environment-specific .env files controlling ports, database paths, and API base URLs. Deployment steps pull the tagged image built and pushed in the packaging stage, then run docker compose pull && docker compose up -d on the target host. A GET /api/health request is issued immediately after deployment; a non-200 response fails the deployment job.

6. Security, Quality Gates, Notifications, and Rollback

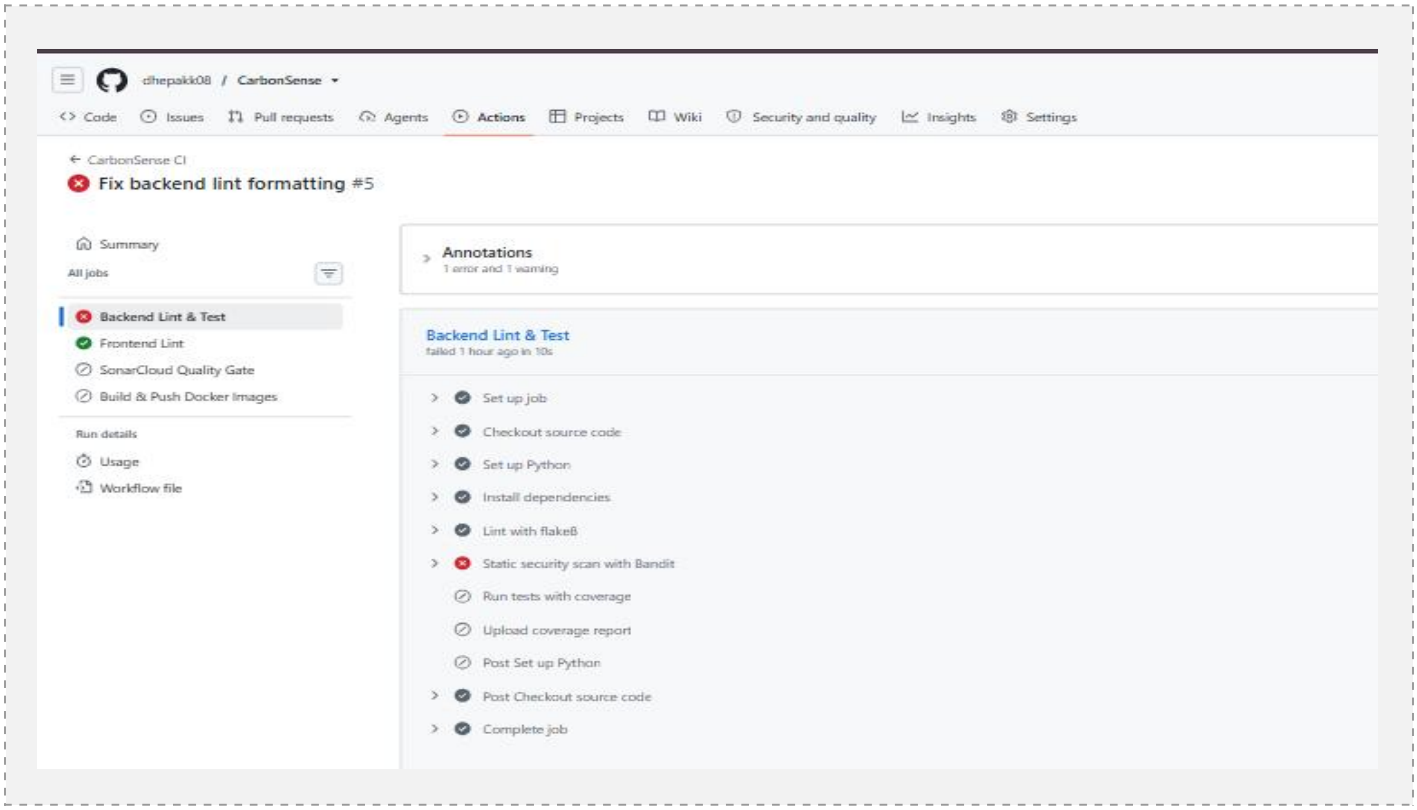
6.1 Security Measures

- All credentials (registry tokens, deployment SSH keys) are stored as encrypted GitHub Secrets, never committed to the repository.
- .env files are excluded via .gitignore; only .env.example templates are versioned.
- bandit performs static security scanning on every backend build, catching issues such as hardcoded secrets or unsafe subprocess/eval calls before code is packaged.

- The GHCR push step uses the automatically scoped, short-lived GITHUB_TOKEN rather than a long-lived personal access token.
- Dependency vulnerability scanning (e.g. pip-audit or Docker Scout) is identified as a recommended future addition to the pipeline.

6.2 Bandit in Practice

During pipeline setup, the bandit step initially failed the build after flagging a real issue in the backend code. The finding was reviewed, the underlying code was corrected, and a subsequent pipeline run passed cleanly — demonstrating the quality gate functioning as intended rather than as a formality.



6.3 Quality Gates

Branch protection rules on main require the CI workflow to pass (backend-test, frontend-lint, and the quality/security jobs) before a pull request can be merged. This prevents untested or non-compliant code from ever reaching the packaging and deployment stages.

6.4 Notifications

A notification step runs at the end of key jobs (build-and-package, deploy-prod) regardless of outcome (if: always()), posting the pipeline status to a Slack channel or email distribution list via a webhook. This gives the team immediate visibility into failures without needing to check the Actions tab manually.

6.5 Rollback Mechanism

Every Docker image is tagged immutably with the triggering commit SHA in addition to latest, so any previously deployed version can be redeployed exactly. In the Production deployment job, if the post-deploy health check fails, a rollback step automatically redeploys the last known-good tag and restarts the stack, minimising downtime without requiring manual intervention.

7. End-to-End Workflow: Commit to Deployment

The following walkthrough traces a single code change through the entire pipeline, referencing the stage numbers in the architecture diagram in Section 2.1.

1. Commit — A developer pushes a commit to a feature branch and opens a pull request into dev.

2. CI Trigger — GitHub Actions detects the push/PR event and starts the CarbonSense CI workflow.

3. Build — Backend dependencies are installed and both Docker images (backend, frontend) are built.

4. Automated Testing — flake8 and pytest run against the backend; ESLint runs against the frontend. Any failure stops the pipeline here.

5. Code Quality & Security — bandit performs a static security scan of the backend codebase.

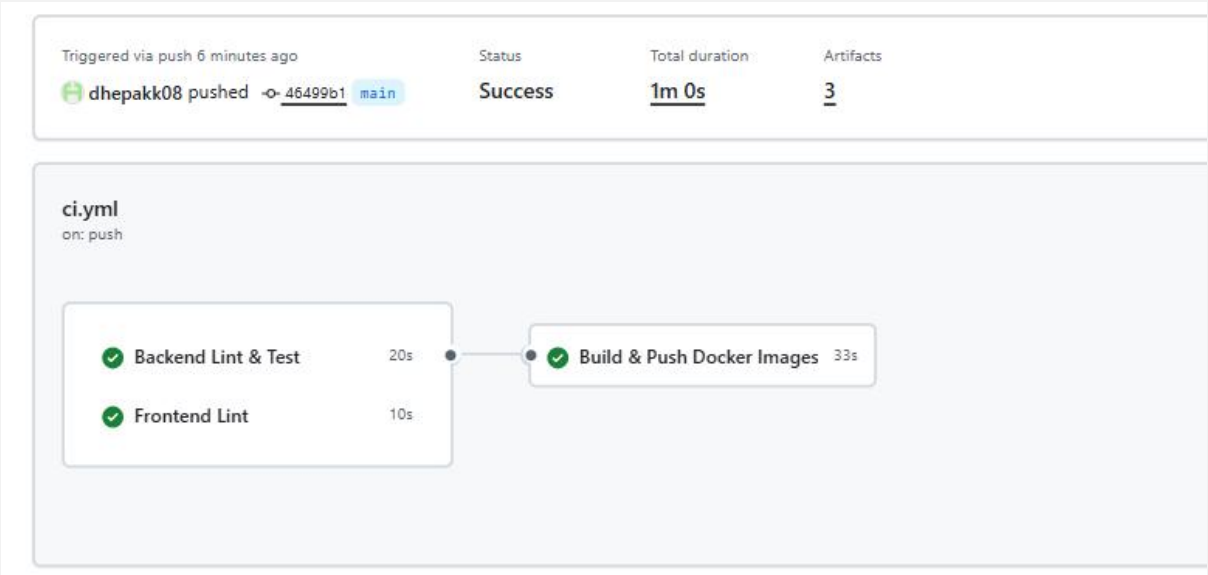
6. Packaging — On success, both Docker images are tagged with the commit SHA and pushed to GHCR.

7a. Deploy to Development — The CD workflow automatically deploys the new images to the Development environment and runs a health check.

7b. Deploy to Staging — Once main is updated (PR merged), CI re-runs and the CD workflow deploys to Staging, pausing for manual reviewer approval first.

7c. Deploy to Production — After staging validation, a second manual approval gate authorises deployment to Production.

8. Monitoring & Rollback — A post-deploy health check confirms the service is healthy; on failure, the pipeline automatically rolls back to the previous stable image tag and notifies the team.



This design demonstrates a complete, low-overhead CI/CD pipeline appropriate for the CarbonSense project: every change is automatically built, tested, and security-scanned before being packaged as a versioned container image and progressively deployed through Development, Staging, and Production with manual approval gates, health checks, and automatic rollback protecting the production environment.